[2026 NS] TLS Connection with Bidirectional Certificates
學生姓名: 杜承諺 學號: 514559010 upload your HW2 to E3 file name: 514559010_report.pdf

Prework on my side

使用 **docker decktop** 建立 **docker**，然後分別建立 **Server** 以及 **Client**

```
PS C:\Users\Yen> docker --version
Docker version 29.3.1, build c2be9cc
PS C:\Users\Yen> docker network create mtls-net
680f126d1cc5d48675d0b3c850c5cccf95ce0275462035251e4c58e0ab9c8faa
PS C:\Users\Yen> docker run -dit --name server --network mtls-net -p 4433:4433 ubuntu:22.04 bash
Unable to find image 'ubuntu:22.04' locally
22.04: Pulling from library/ubuntu
f63eb04151bc: Pull complete
bf0d6867143e: Download complete
Digest: sha256:962f6cadeae0ea6284001009daa4cc9a8c37e75d1f5191cf0eb83fe565b63dd7
Status: Downloaded newer image for ubuntu:22.04
986a38842317566b127c0ac2ef8ec7835d3007906649d9df1adce789bf0f54f7
PS C:\Users\Yen> docker run -dit --name client --network mtls-net ubuntu:22.04 bash
4b1939485ade0a032b529b0047b59626f6876000c7ed113b0f6ac6ed9c005f21
PS C:\Users\Yen> docker ps
CONTAINER ID   IMAGE          COMMAND   CREATED          STATUS          PORTS                                       NAMES
4b1939485ade   ubuntu:22.04   "bash"    9 seconds ago    Up 8 seconds                                                client
986a38842317   ubuntu:22.04   "bash"    24 seconds ago   Up 23 seconds   0.0.0.0:4433->4433/tcp, [::]:4433->4433/tcp   server
```

```
root@986a38842317: /pki        Windows PowerShell
Running hooks in /etc/ca-certificates/update.d...
done.
root@4b1939485ade:/# exit
exit
PS C:\Users\Yen> docker exec -it server bash
root@986a38842317:/# ls -l
total 64
lrwxrwxrwx   1 root root    7 Apr 10 02:21 bin -> usr/bin
drwxr-xr-x   2 root root 4096 Apr 18  2022 boot
drwxr-xr-x   5 root root  360 Apr 19 09:04 dev
drwxr-xr-x   1 root root 4096 Apr 19 09:07 etc
drwxr-xr-x   2 root root 4096 Apr 18  2022 home
lrwxrwxrwx   1 root root    7 Apr 10 02:21 lib -> usr/lib
lrwxrwxrwx   1 root root    9 Apr 10 02:21 lib32 -> usr/lib32
lrwxrwxrwx   1 root root    9 Apr 10 02:21 lib64 -> usr/lib64
lrwxrwxrwx   1 root root   10 Apr 10 02:21 libx32 -> usr/libx32
drwxr-xr-x   2 root root 4096 Apr 10 02:21 media
drwxr-xr-x   2 root root 4096 Apr 10 02:21 mnt
drwxr-xr-x   2 root root 4096 Apr 10 02:21 opt
dr-xr-xr-x 367 root root    0 Apr 19 09:04 proc
drwx------   1 root root 4096 Apr 19 09:07 root
drwxr-xr-x   1 root root 4096 Apr 19 09:07 run
lrwxrwxrwx   1 root root    8 Apr 10 02:21 sbin -> usr/sbin
drwxr-xr-x   2 root root 4096 Apr 10 02:21 srv
dr-xr-xr-x  13 root root    0 Apr 19 09:04 sys
drwxrwxrwt   1 root root 4096 Apr 19 09:07 tmp
drwxr-xr-x   1 root root 4096 Apr 10 02:21 usr
drwxr-xr-x   1 root root 4096 Apr 10 02:31 var
```

Part 1: PKI Construction (30 pts)

List all **openssl** commands used to generate the CA, keys, CSRs (Certificate Signing Requests), and Certificates.

CA 建立 > 生成 key(RSA)

openssl genrsa -out rootCA.key 4096

```
root@986a38842317: /pki        Windows PowerShell
drwxrwxrwt   1 root root 4096 Apr 19 09:07 tmp
drwxr-xr-x   1 root root 4096 Apr 10 02:21 usr
drwxr-xr-x   1 root root 4096 Apr 10 02:31 var
root@986a38842317:/# mkdir -p/pki && cd /pki
mkdir: invalid option -- '/'
Try 'mkdir --help' for more information.
root@986a38842317:/# mkdir -p /pki && cd /pki
root@986a38842317:/pki# openssl genrsa -out rootCA.key 4096
root@986a38842317:/pki# openssl req -x509 -new -nodes -key rootCA.key -sha256 -days 3650 \
> -subj "/C=TW/ST=Taoyuan/L=Taoyuan/O=NS2026/OU=RootCA/CN=MyRootCA" \
> -out rootCA.crt
root@986a38842317:/pki# ls -l
total 8
-rw-r--r-- 1 root root 2029 Apr 19 09:14 rootCA.crt
-rw------- 1 root root 3272 Apr 19 09:11 rootCA.key
root@986a38842317:/pki# ls -la
total 16
drwxr-xr-x 2 root root 4096 Apr 19 09:14 .
drwxr-xr-x 1 root root 4096 Apr 19 09:10 ..
-rw-r--r-- 1 root root 2029 Apr 19 09:14 rootCA.crt
-rw------- 1 root root 3272 Apr 19 09:11 rootCA.key
root@986a38842317:/pki# openssl genrsa -out server.key 2048
root@986a38842317:/pki# openssl req -new -key server.key \
> -subj "/C=TW/ST=Taoyuan/L=Taoyuan/O=NS2026/OU=Server/CN=localhost" \
> -out server.csr
root@986a38842317:/pki# ls -l
total 16
-rw-r--r-- 1 root root 2029 Apr 19 09:14 rootCA.crt
-rw------- 1 root root 3272 Apr 19 09:11 rootCA.key
-rw-r--r-- 1 root root 1005 Apr 19 09:17 server.csr
-rw------- 1 root root 1704 Apr 19 09:15 server.key
root@986a38842317:/pki# cat > server_ext.cnf <<EOF
> subjectAltName = DNS:localhost, IP:127.0.0.1
> extendedKeyUsage = serverAuth
> EOF
root@986a38842317:/pki# openssl x509 -req -in server.csr \
> -CA rootCA.crt -CAkey rootCA.key -CAcreateserial \
> -out server.crt -days 365 -sha256 \
> -extfile server_ext.cnf
Certificate request self-signature ok
subject=C = TW, ST = Taoyuan, L = Taoyuan, O = NS2026, OU = Server, CN = localhost
root@986a38842317:/pki# openssl genrsa -out client.key 2048
root@986a38842317:/pki# openssl req -new -key client.key \
  -subj "/C=TW/ST=Taoyuan/L=Taoyuan/O=NS2026/OU=Client/CN=MyClient" \
  -out client.csr
root@986a38842317:/pki# cat > client_ext.cnf <<EOF
```

用私鑰簽名，產生 Root CA 憑證

**openssl req -x509 -new -nodes -key rootCA.key -sha256 -days 3650 **
-subj
"/C=TW/ST=Taoyuan/L=Taoyuan/O=NS2026/OU=RootCA/CN=MyRootCA"
\ -out rootCA.crt

做 Server 斯要

openssl genrsa -out server.key 2048

```
-rw------- 1 root root 3272 Apr 19 09:11 rootCA.key
root@986a38842317:/pki# ls -la
total 16
drwxr-xr-x 2 root root 4096 Apr 19 09:14 .
drwxr-xr-x 1 root root 4096 Apr 19 09:10 ..
-rw-r--r-- 1 root root 2029 Apr 19 09:14 rootCA.crt
-rw------- 1 root root 3272 Apr 19 09:11 rootCA.key
root@986a38842317:/pki# openssl genrsa -out server.key 2048
root@986a38842317:/pki# openssl req -new -key server.key \
> -subj "/C=TW/ST=Taoyuan/L=Taoyuan/O=NS2026/OU=Server/CN=localhost" \
> -out server.csr
root@986a38842317:/pki# ls -l
total 16
-rw-r--r-- 1 root root 2029 Apr 19 09:14 rootCA.crt
-rw------- 1 root root 3272 Apr 19 09:11 rootCA.key
-rw-r--r-- 1 root root 1005 Apr 19 09:17 server.csr
-rw------- 1 root root 1704 Apr 19 09:15 server.key
root@986a38842317:/pki# cat > server_ext.cnf <<EOF
> subjectAltName = DNS:localhost, IP:127.0.0.1
> extendedKeyUsage = serverAuth
> EOF
```

產生 Server 的 CSR（申請表）

**openssl req -new -key server.key **

-subj

**"/C=TW/ST=Taoyuan/L=Taoyuan/O=NS2026/OU=Server/CN=localhost" **

-out server.csr

```
-rw------- 1 root root 3272 Apr 19 09:11 rootCA.key
root@986a38842317:/pki# openssl genrsa -out server.key 2048
root@986a38842317:/pki# openssl req -new -key server.key \
> -subj "/C=TW/ST=Taoyuan/L=Taoyuan/O=NS2026/OU=Server/CN=localhost" \
> -out server.csr
root@986a38842317:/pki# ls -l
total 16
-rw-r--r-- 1 root root 2029 Apr 19 09:14 rootCA.crt
-rw------- 1 root root 3272 Apr 19 09:11 rootCA.key
-rw-r--r-- 1 root root 1005 Apr 19 09:17 server.csr
-rw------- 1 root root 1704 Apr 19 09:15 server.key
root@986a38842317:/pki# cat > server_ext.cnf <<EOF
> subjectAltName = DNS:localhost, IP:127.0.0.1
> extendedKeyUsage = serverAuth
> EOF
```

建立 SAN 擴充檔

cat > server_ext.cnf <<EOF

subjectAltName = DNS:localhost, IP:127.0.0.1

extendedKeyUsage = serverAuth EOF

用 Root CA 簽發 Server 憑證

**openssl x509 -req -in server.csr **

**-CA rootCA.crt -CAkey rootCA.key -CAcreateserial **

-out server.crt -days 365 -sha256 \ -extfile server_ext.cnf

```
-rw------- 1 root root 1704 Apr 19 09:15 server.key
root@986a38842317:/pki# cat > server_ext.cnf <<EOF
> subjectAltName = DNS:localhost, IP:127.0.0.1
> extendedKeyUsage = serverAuth
> EOF
root@986a38842317:/pki# openssl x509 -req -in server.csr \
> -CA rootCA.crt -CAkey rootCA.key -CAcreateserial \
> -out server.crt -days 365 -sha256 \
> -extfile server_ext.cnf
Certificate request self-signature ok
subject=C = TW, ST = Taoyuan, L = Taoyuan, O = NS2026, OU = Server, CN = localhost
```

做 Client 的私鑰

openssl genrsa -out client.key 2048

產生 Client 的 CSR

**openssl req -new -key client.key **

-subj

**"/C=TW/ST=Taoyuan/L=Taoyuan/O=NS2026/OU=Client/CN=MyClient" **

-out client.csr

```
subject=C = TW, ST = Taoyuan, L = Taoyuan, O = NS2026, OU = Server, CN = localhost
root@986a38842317:/pki# openssl genrsa -out client.key 2048
root@986a38842317:/pki# openssl req -new -key client.key \
  -subj "/C=TW/ST=Taoyuan/L=Taoyuan/O=NS2026/OU=Client/CN=MyClient" \
  -out client.csr
root@986a38842317:/pki# cat > client_ext.cnf <<EOF
   extendedKeyUsage = clientAuth
EOF
root@986a38842317:/pki# openssl x509 -req -in client.csr \
> -CA rootCA.crt -CAkey rootCA.key -CAcreateserial \
> -out client.crt -days 365 -sha256 \
> -extfile client_ext.cnf
Certificate request self-signature ok
subject=C = TW, ST = Taoyuan, L = Taoyuan, O = NS2026, OU = Client, CN = MyClient
```

建立 Client 的 SAN 擴充檔

cat > client_ext.cnf <<EOF

extendedKeyUsage = clientAuth

EOF

```
root@986a38842317:/pki# openssl req -new -key client.key \
  -subj "/C=TW/ST=Taoyuan/L=Taoyuan/O=NS2026/OU=Client/CN=My
  -out client.csr
root@986a38842317:/pki# cat > client_ext.cnf <<EOF
   extendedKeyUsage = clientAuth
EOF
```

用 Root CA 簽發 Client 憑證

**openssl x509 -req -in client.csr **

**-CA rootCA.crt -CAkey rootCA.key -CAcreateserial **

**-out client.crt -days 365 -sha256 **

-extfile client_ext.cnf

```
EOF
root@986a38842317:/pki# openssl x509 -req -in client.csr \
> -CA rootCA.crt -CAkey rootCA.key -CAcreateserial \
> -out client.crt -days 365 -sha256 \
> -extfile client_ext.cnf
Certificate request self-signature ok
subject=C = TW, ST = Taoyuan, L = Taoyuan, O = NS2026, OU = Client, CN = MyClient
```

檢查 Server 憑證內容

openssl x509 -in server.crt -text -noout | head -20

```
subject=C = TW, ST = Taoyuan, L = Taoyuan, O = NS2026, OU = Client, CN = MyClient
root@986a38842317:/pki# openssl x509 -in server.crt -text -noout | head -20
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            21:5a:a5:55:7b:af:d5:3a:55:3e:4a:84:a2:31:5b:fc:c1:27:d4:c6
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C = TW, ST = Taoyuan, L = Taoyuan, O = NS2026, OU = RootCA, CN = MyRootCA
        Validity
            Not Before: Apr 19 09:21:44 2026 GMT
            Not After : Apr 19 09:21:44 2027 GMT
        Subject: C = TW, ST = Taoyuan, L = Taoyuan, O = NS2026, OU = Server, CN = localhost
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (2048 bit)
                Modulus:
                    00:90:40:c5:9f:b8:58:85:65:6b:78:24:cc:54:6f:
                    4a:d4:cc:aa:e7:4b:5f:c6:14:1a:e1:e4:89:b2:cf:
                    9f:0a:cf:b9:85:70:22:26:79:eb:d5:45:96:4d:6a:
                    dd:15:f6:1b:d2:58:87:a2:41:64:7b:33:8f:b8:43:
                    56:94:c3:02:50:62:22:27:13:ee:6a:c7:f3:bc:59:
root@986a38842317:/pki# openssl verify -CAfile rootCA.crt server.crt
server.crt: OK
root@986a38842317:/pki# openssl verify -CAfile rootCA.crt client.crt
client.crt: OK
```

Part 2: Service Configuration (20 pts)

Provide the exact content of your **stunnel.conf** file

啟動後端明文 HTTP (port: 8000)

nohup python3 -m http.server 8000 > /tmp/http.log 2>&1 &

```
root@986a38842317:/pki# nohup python3 -m http.server 8000 > /tmp/http.log 2>&1 &
[1] 3898
root@986a38842317:/pki# curl http://localhost:8000
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8">
<title>Directory listing for /</title>
</head>
<body>
<h1>Directory listing for /</h1>
<hr>
<ul>
<li><a href="client.crt">client.crt</a></li>
<li><a href="client.csr">client.csr</a></li>
<li><a href="client.key">client.key</a></li>
<li><a href="client_ext.cnf">client_ext.cnf</a></li>
<li><a href="rootCA.crt">rootCA.crt</a></li>
<li><a href="rootCA.key">rootCA.key</a></li>
<li><a href="server.crt">server.crt</a></li>
<li><a href="server.csr">server.csr</a></li>
<li><a href="server.key">server.key</a></li>
<li><a href="server_ext.cnf">server_ext.cnf</a></li>
</ul>
<hr>
</body>
</html>
root@986a38842317:/pki#
```

stunnel.conf 設定檔

```
</body>
</html>
root@986a38842317:/pki# cat > /etc/stunnel/stunnel.conf <<'EOF'
; ========================================================
; stunnel4 mTLS Proxy Configuration
; Course: 2026 Network Security
; Purpose: Wrap plaintext HTTP (port 8000) with mTLS on port 4433
; ========================================================

; ---------- Global Options ----------
pid = /var/run/stunnel4/stunnel.pid
output = /var/log/stunnel4/stunnel.log
debug = 5
foreground = no

; ---------- mTLS Service Definition ----------
[mtls-https]
; Listen on all interfaces, port 4433 (TLS side)
accept = 0.0.0.0:4433

; Forward decrypted traffic to local plaintext HTTP server
connect = 127.0.0.1:8000

; Act as TLS server
client = no

; --- Server identity ---
cert = /pki/server.crt
key  = /pki/server.key

; --- Client certificate verification (mTLS core) ---
; verify = 2 means: require and verify client certificate against CAfile
CAfile = /pki/rootCA.crt
verify = 2

; --- TLS protocol hardening ---
sslVersion = TLSv1.2
options = NO_SSLv2
options = NO_SSLv3
EOF
root@986a38842317:/pki#
```

設定項	功能
pid	stunnel4 程式的 PID 檔位置
output	日誌輸出路徑，方便除錯
debug = 5	日誌詳細程度（1–7，數字越大越詳細）
foreground = no	以背景守護模式執行
[mtls-https]	服務名稱，可自訂
accept = 0.0.0.0:4433	監聽所有網卡的 4433 埠
connect = 127.0.0.1:8000	驗證通過後，把流量轉發到後端
client = no	本服務作為 TLS Server（不是 Client）
cert / key	Server 的身分證和私鑰
CAfile	用這個 CA 驗證客戶端憑證
verify = 2	**強制要求並驗證 Client 憑證**
sslVersion	指定 TLS 版本

Verify 等級

0 不驗證

1 如果對方出示憑證才驗證

2 要求對方出示憑證且必須通過驗證

3 驗證且檢查本地快取

4 不查 CA，只看本地憑證

啟動 stunnel4

```
root@986a38842317: /pki    root@986a38842317: /pki    root@4b1939485ade: /
Processing triggers for libc-bin (2.35-0ubuntu3.13) ...
root@986a38842317:/pki# ss -tlnp | grep 4433
LISTEN 0      4096         0.0.0.0:4433       0.0.0.0:*    users:(("stunnel4",pid=3909,fd=8))
root@986a38842317:/pki# ps aux | grep stunnel
root        3909  0.0  0.0  88592  4164 ?        Ssl  11:37   0:00 stunnel4 /etc/stunnel/stunnel.conf
root        4069  0.0  0.0   3472  1600 pts/2    S+   11:38   0:00 grep --color=auto stunnel
root@986a38842317:/pki# curl -v https://localhost:4433/ \
  --cacert /pki/rootCA.crt \
  --cert /pki/client.crt \
  --key /pki/client.key
*   Trying 127.0.0.1:4433...
* Connected to localhost (127.0.0.1) port 4433 (#0)
* ALPN, offering h2
* ALPN, offering http/1.1
*  CAfile: /pki/rootCA.crt
*  CApath: /etc/ssl/certs
* TLSv1.0 (OUT), TLS header, Certificate Status (22):
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.2 (IN), TLS handshake, Certificate (11):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.2 (IN), TLS handshake, Server key exchange (12):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.2 (IN), TLS handshake, Request CERT (13):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.2 (IN), TLS handshake, Server finished (14):
* TLSv1.2 (OUT), TLS header, Certificate Status (22):
* TLSv1.2 (OUT), TLS handshake, Certificate (11):
* TLSv1.2 (OUT), TLS header, Certificate Status (22):
* TLSv1.2 (OUT), TLS handshake, Client key exchange (16):
* TLSv1.2 (OUT), TLS header, Certificate Status (22):
* TLSv1.2 (OUT), TLS handshake, CERT verify (15):
* TLSv1.2 (OUT), TLS header, Finished (20):
* TLSv1.2 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.2 (OUT), TLS header, Certificate Status (22):
* TLSv1.2 (OUT), TLS handshake, Finished (20):
* TLSv1.2 (IN), TLS header, Finished (20):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.2 (IN), TLS handshake, Finished (20):
* SSL connection using TLSv1.2 / ECDHE-RSA-AES256-GCM-SHA384
* ALPN, server did not agree to a protocol
* Server certificate:
*  subject: C=TW; ST=Taoyuan; L=Taoyuan; O=NS2026; OU=Server; CN=localhost
*  start date: Apr 19 09:21:44 2026 GMT
*  expire date: Apr 19 09:21:44 2027 GMT
*  subjectAltName: host "localhost" matched cert's "localhost"
*  issuer: C=TW; ST=Taoyuan; L=Taoyuan; O=NS2026; OU=RootCA; CN=MyRootCA
*  SSL certificate verify ok.
* TLSv1.2 (OUT), TLS header, Supplemental data (23):
> GET / HTTP/1.1
> Host: localhost:4433
> User-Agent: curl/7.81.0
> Accept: */*
>
* TLSv1.2 (IN), TLS header, Supplemental data (23):
* Mark bundle as not supporting multiuse
* HTTP 1.0, assume close after body
```

Part 3: Connection Testing (30 pts)

Provide screenshots of the curl -v (verbose) output for the following three scenarios:

1. Success: Connection using the valid client certificate and CA (. curl -v --cacert ... -- cert ... --key ...). Show the HTTP 200 OK response

成功連線 mTLS

**curl -v https://server:4433/ **
**--cacert /pki/rootCA.crt **
**--cert /pki/client.crt **
--key /pki/client.key

```
root@986a38842317: /pki    root@986a38842317: /pki    root@4b1939485ade: /

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Yen> docker exec -it client bash
root@4b1939485ade:/# curl -v https://server:4433/ \
  --cacert /pki/rootCA.crt \
  --cert /pki/client.crt \
  --key /pki/client.key
*   Trying 172.18.0.2:4433...
* Connected to server (172.18.0.2) port 4433 (#0)
* ALPN, offering h2
* ALPN, offering http/1.1
*  CAfile: /pki/rootCA.crt
*  CApath: /etc/ssl/certs
* TLSv1.0 (OUT), TLS header, Certificate Status (22):
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.2 (IN), TLS handshake, Certificate (11):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.2 (IN), TLS handshake, Server key exchange (12):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.2 (IN), TLS handshake, Request CERT (13):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.2 (IN), TLS handshake, Server finished (14):
* TLSv1.2 (OUT), TLS header, Certificate Status (22):
* TLSv1.2 (OUT), TLS handshake, Certificate (11):
* TLSv1.2 (OUT), TLS header, Certificate Status (22):
* TLSv1.2 (OUT), TLS handshake, Client key exchange (16):
* TLSv1.2 (OUT), TLS header, Certificate Status (22):
* TLSv1.2 (OUT), TLS handshake, CERT verify (15):
* TLSv1.2 (OUT), TLS header, Finished (20):
* TLSv1.2 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.2 (OUT), TLS header, Certificate Status (22):
* TLSv1.2 (OUT), TLS handshake, Finished (20):
* TLSv1.2 (IN), TLS header, Finished (20):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.2 (IN), TLS handshake, Finished (20):
* SSL connection using TLSv1.2 / ECDHE-RSA-AES256-GCM-SHA384
* ALPN, server did not agree to a protocol
* Server certificate:
*  subject: C=TW; ST=Taoyuan; L=Taoyuan; O=NS2026; OU=Server; CN=localhost
*  start date: Apr 19 09:21:44 2026 GMT
*  expire date: Apr 19 09:21:44 2027 GMT
*  subjectAltName does not match server
* SSL: no alternative certificate subject name matches target host name 'server'
* Closing connection 0
* TLSv1.2 (OUT), TLS header, Unknown (21):
* TLSv1.2 (OUT), TLS alert, close notify (256):
curl: (60) SSL: no alternative certificate subject name matches target host name 'server'
More details here: https://curl.se/docs/sslcerts.html

curl failed to verify the legitimacy of the server and therefore could not
establish a secure connection to it. To learn more about this situation and
how to fix it, please visit the web page mentioned above.
root@4b1939485ade:/#
```

2. Failure 1: Connection without providing a client certificate. Show the TLS alert/handshake failure.

不提供 Client 憑證

**curl -v https://localhost:4433/ **
** --resolve localhost:4433:$(getent hosts server | awk '{print $1}') **
** --cacert /pki/rootCA.crt**

```
root@4b1939485ade:/# curl -v https://localhost:4433/ \
  --resolve localhost:4433:$(getent hosts server | awk '{print $1}') \
  --cacert /pki/rootCA.crt
* Added localhost:4433:172.18.0.2 to DNS cache
* Hostname localhost was found in DNS cache
*   Trying 172.18.0.2:4433...
* Connected to localhost (172.18.0.2) port 4433 (#0)
* ALPN, offering h2
* ALPN, offering http/1.1
*  CAfile: /pki/rootCA.crt
*  CApath: /etc/ssl/certs
* TLSv1.0 (OUT), TLS header, Certificate Status (22):
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.2 (IN), TLS handshake, Certificate (11):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.2 (IN), TLS handshake, Server key exchange (12):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.2 (IN), TLS handshake, Request CERT (13):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.2 (IN), TLS handshake, Server finished (14):
* TLSv1.2 (OUT), TLS header, Certificate Status (22):
* TLSv1.2 (OUT), TLS handshake, Certificate (11):
* TLSv1.2 (OUT), TLS header, Certificate Status (22):
* TLSv1.2 (OUT), TLS handshake, Client key exchange (16):
* TLSv1.2 (OUT), TLS header, Finished (20):
* TLSv1.2 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.2 (OUT), TLS header, Certificate Status (22):
* TLSv1.2 (OUT), TLS handshake, Finished (20):
* TLSv1.2 (IN), TLS header, Unknown (21):
* TLSv1.2 (IN), TLS alert, handshake failure (552):
* error:0A000410:SSL routines::sslv3 alert handshake failure
* Closing connection 0
curl: (35) error:0A000410:SSL routines::sslv3 alert handshake failure
root@4b1939485ade:/#
```

3. Failure 2: Connection using **an unauthorized client certificate** (generate a fake cert signed by a different, untrusted CA). Show the server rejecting the connection

用未授權 CA 簽的 Client 憑證

先在 Client 容器內做一組「假的 CA + 假的 Client 憑證」

mkdir -p /fake-pki && cd /fake-pki

 # 1. 造一個假的 Root CA

openssl genrsa -out fakeCA.key 4096

**openssl req -x509 -new -nodes -key fakeCA.key -sha256 -days 365 **
** -subj "/C=TW/O=FakeCA/CN=EvilRootCA" **

-out fakeCA.crt

 # 2. 用假 CA 簽一張假 Client 憑證

openssl genrsa -out fake-client.key 2048

**openssl req -new -key fake-client.key **

**-subj "/C=TW/O=FakeCA/CN=EvilClient" **

**-out fake-client.csr openssl x509 -req -in fake-client.csr **

**-CA fakeCA.crt -CAkey fakeCA.key -CAcreateserial **

-out fake-client.crt -days 365 -sha256

```
Organization Name (eg, company) [Internet Widgits Pty Ltd]:^C
root@4b1939485ade:/fake-pki# openssl genrsa -out fakeCA.key 4096
openssl req -x509 -new -nodes -key fakeCA.key -sha256 -days 365 \
  -subj "/C=TW/O=FakeCA/CN=EvilRootCA" \
  -out fakeCA.crt
root@4b1939485ade:/fake-pki# openssl genrsa -out fake-client.key 2048
openssl req -new -key fake-client.key \
  -subj "/C=TW/O=FakeCA/CN=EvilClient" \
  -out fake-client.csr
openssl x509 -req -in fake-client.csr \
  -CA fakeCA.crt -CAkey fakeCA.key -CAcreateserial \
  -out fake-client.crt -days 365 -sha256
Certificate request self-signature ok
subject=C = TW, O = FakeCA, CN = EvilClient
root@4b1939485ade:/fake-pki# curl -v https://localhost:4433/ \
  --resolve localhost:4433:$(getent hosts server | awk '{print $1}') \
  --cacert /pki/rootCA.crt \
  --cert /fake-pki/fake-client.crt \
```

測試連線

**curl -v https://localhost:4433/ **

**--resolve localhost:4433:$(getent hosts server | awk '{print $1}') **

**--cacert /pki/rootCA.crt \ --cert /fake-pki/fake-client.crt **

--key /fake-pki/fake-client.key

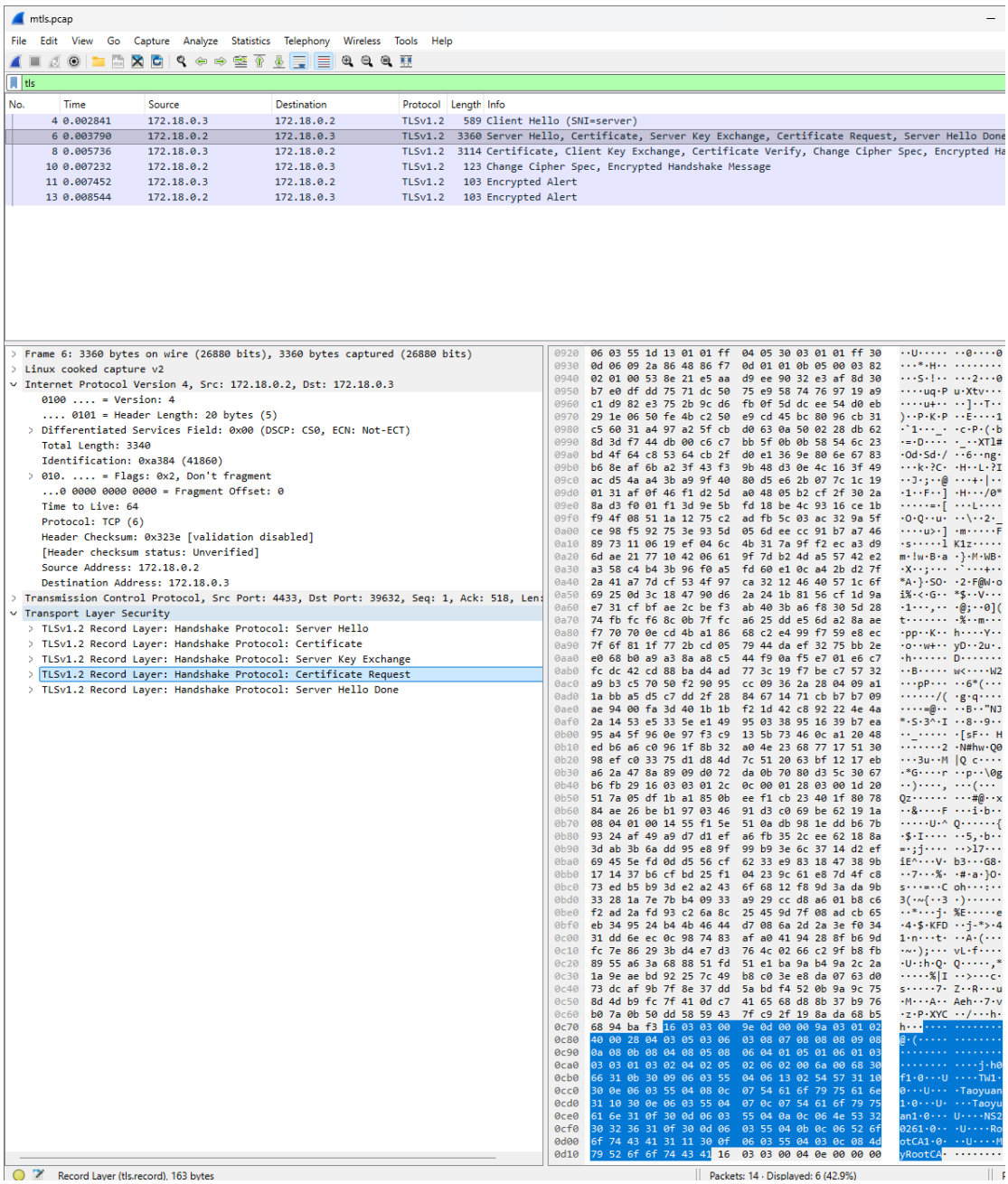
```
root@4b1939485ade:/fake-pki# curl -v https://localhost:4433/ \
  --resolve localhost:4433:$(getent hosts server | awk '{print $1}') \
  --cacert /pki/rootCA.crt \
  --cert /fake-pki/fake-client.crt \
  --key /fake-pki/fake-client.key
* Added localhost:4433:172.18.0.2 to DNS cache
* Hostname localhost was found in DNS cache
*   Trying 172.18.0.2:4433...
* Connected to localhost (172.18.0.2) port 4433 (#0)
* ALPN, offering h2
* ALPN, offering http/1.1
*  CAfile: /pki/rootCA.crt
*  CApath: /etc/ssl/certs
* TLSv1.0 (OUT), TLS header, Certificate Status (22):
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.2 (IN), TLS handshake, Certificate (11):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.2 (IN), TLS handshake, Server key exchange (12):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.2 (IN), TLS handshake, Request CERT (13):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.2 (IN), TLS handshake, Server finished (14):
* TLSv1.2 (OUT), TLS header, Certificate Status (22):
* TLSv1.2 (OUT), TLS handshake, Certificate (11):
* TLSv1.2 (OUT), TLS header, Certificate Status (22):
* TLSv1.2 (OUT), TLS handshake, Client key exchange (16):
* TLSv1.2 (OUT), TLS header, Certificate Status (22):
* TLSv1.2 (OUT), TLS handshake, CERT verify (15):
* TLSv1.2 (OUT), TLS header, Finished (20):
* TLSv1.2 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.2 (OUT), TLS header, Certificate Status (22):
* TLSv1.2 (OUT), TLS handshake, Finished (20):
* TLSv1.2 (IN), TLS header, Unknown (21):
* TLSv1.2 (IN), TLS alert, unknown CA (560):
* error:0A000418:SSL routines::tlsv1 alert unknown ca
* Closing connection 0
curl: (35) error:0A000418:SSL routines::tlsv1 alert unknown ca
root@4b1939485ade:/fake-pki#
```

Part 4: Packet Analysis (20 pts)

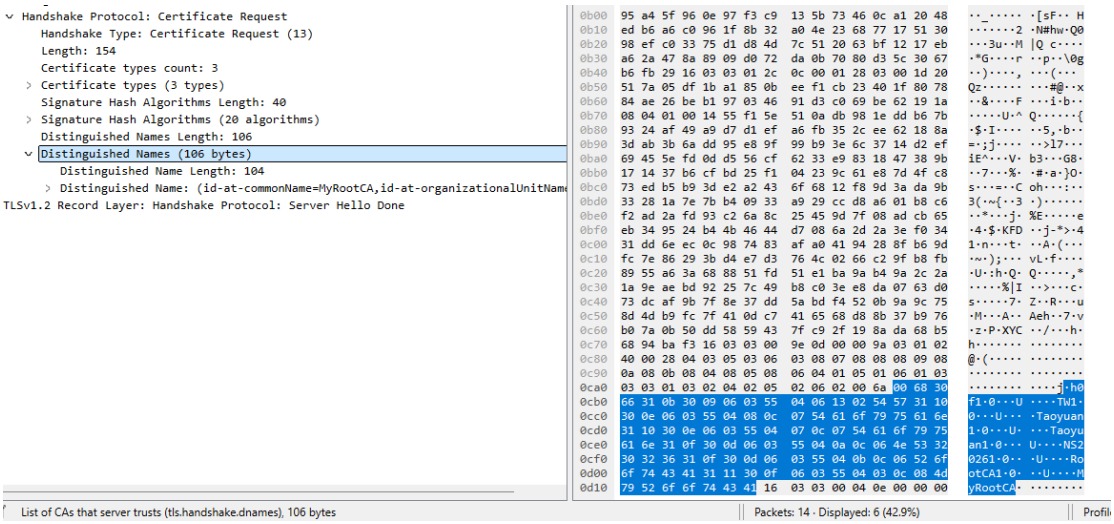
Provide a Wireshark or tcpdump screenshot of the network traffic during the successful mTLS handshake (Scenario 1)

```
root@986a38842317:/# tcpdump -i any -w /tmp/mtls.pcap 'port 4433' -v
tcpdump: data link type LINUX_SLL2
tcpdump: listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
^C14 packets captured
14 packets received by filter
0 packets dropped by kernel
root@986a38842317:/#
```

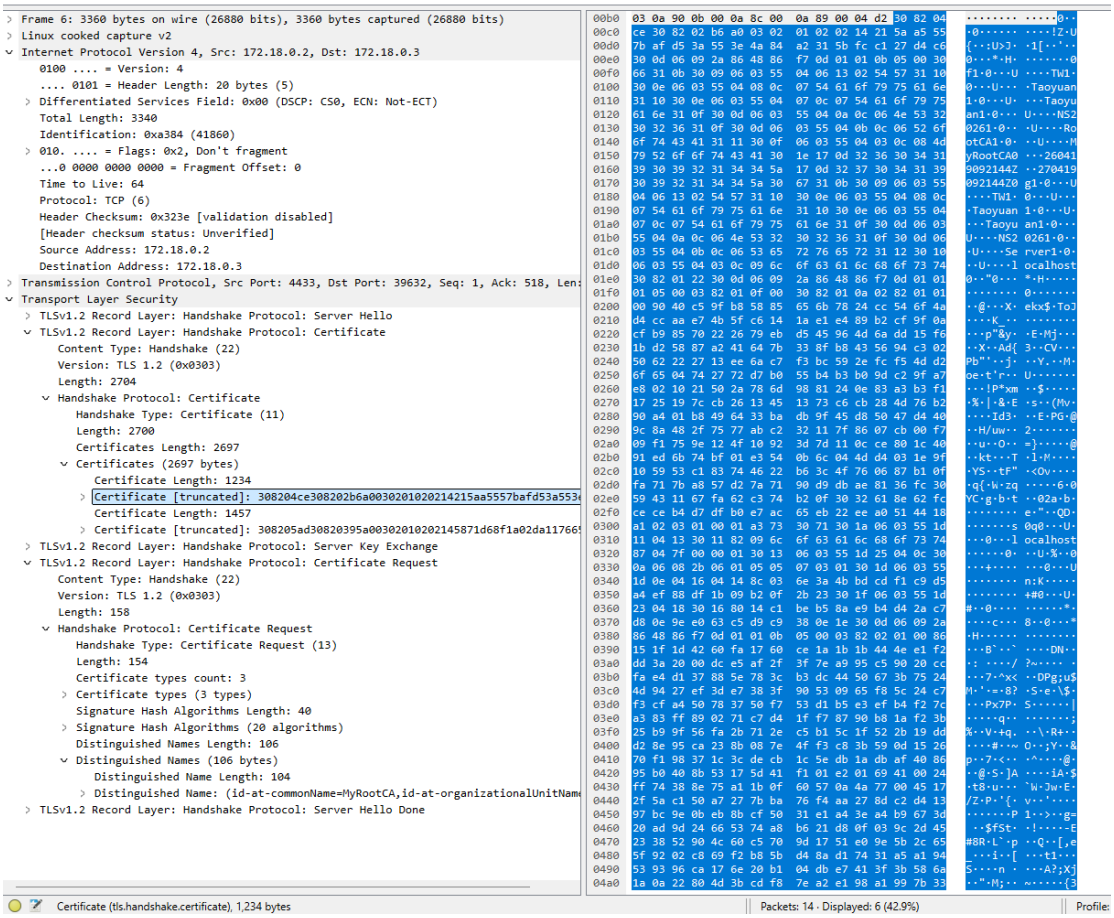
以 **Wireshark** 開啟，然後在 **Filter** 選擇 **TLS**，並選擇 **Certificate Quest** 封包觀察，點開 **TLS**



可以找到 **Distinguished Names**，會顯示信任的 **CA**



接下來，觀察 Client 端的 Certificate 封包



個人資料整理討論：

Q: verify=2 vs verify=1 在 TLS 的 verify 等級

- 因為在 level 1 (verify=1)：代表有憑證即信任。驗證對等方提供的憑證，但如果對等方沒有提供憑證，則允許連接（憑證無效亦可能允許）。
- 而在 level 2 (verify=2)：代表嚴格 CA 驗證。這是常見的生產環境設置，要求對等方必須提供憑證，且必須經由本地信任的 CA (CAfile) 簽署。
- 本次作業為測試自己的 PKI，因此為使用 CA，也就是符合 verify2 的驗證流程，因此設定為 verify=2。

Q: 為什麼 mTLS 比單向 TLS 更安全？

在傳統 HTTPS 中，只有 Server 需要證明身分，Client 是匿名的，因此任何知道 URL 的人都能發起請求。mTLS 則強制 Client 也出示由可信 CA 簽發的憑證，將「誰能連線」的控制權從應用層（帳號密碼）下放到傳輸層，大幅降低暴力破解與未授權存取的風險。
在微服務架構中，mTLS 是 Zero Trust 的基礎建設之一。

因為本次是使用 Docker 進行憑證的認證，因此將憑證放在 docker 容器內。然而在正常狀況中，容器本身應該是要是無狀態(stateless)且可丟棄(disposable)的，在生產環境中，這種做法不符合 Immutable Infrastructure 原則——一旦容器被重建，憑證將遺失。
較安全的做法可以將憑證放在 Docker Volume 或 Kubernetes Secret 等工具，然後以掛載方式提供 Stunnel4，實現「容器無狀態 + 敏感資料集中管理」。